

🧐 RP - 323 - Programmation fonctionnelle

[!TIP] > **Référence Javascript:** <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference>

Tester du code JS : <https://runjs.app/play>

Convertir en PDF : <https://marketplace.visualstudio.com/items?itemName=manuth.markdown-converter>

Table des matières

- Introduction
- Opérateurs javascript super-coool 😎
 - opérateur `?:`
 - opérateur `??`
 - opérateur `??=`
 - opérateur de décomposition 'spread' `...`
 - Déstructuration
- Date et Heure
 - Obtenir la date et/ou heure actuelle
- Math
 - `Math.PI` - la constante π
 - `Math.abs()` - la |valeur absolue| d'un nombre
 - `Math.pow()` - élever à une puissance
 - `Math.min()` - plus petite valeur
 - `Math.max()` - plus grande valeur
 - `Math.ceil()` - arrondir à la prochaine valeur entière la plus proche
 - `Math.floor()` - arrondir à la précédente valeur entière la plus proche
 - `Math.round()` - arrondir à la valeur entière la plus proche
 - `Math.trunc()` - supprime la virgule et retourne la partie entière d'un nombre
 - `Math.sqrt()` - la racine carrée d'un nombre
 - `Math.random()` - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)
- JSON
 - `JSON.stringify()` - transformer un objet Javascript en JSON
 - `JSON.parse()` - transformer du JSON en objet Javascript
- Chaînes de caractères
 - `split()` - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau
 - `trim()`, `trimStart()` et `trimEnd()` - épuration des espaces en trop dans une chaîne (trimming)
 - `padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères
- Console
 - `console.log()` - Afficher un message sur la console
 - `console.info()`, `warn()` et `error()` - Afficher un message sur la console (filtrables)

- `console.table()` - Afficher tout un tableau ou un objet sur la console
- `console.time()`, `timeLog()` et `timeEnd()` - Chronométrer une durée d'exécution
- Tableaux
 - `forEach` - parcourir les éléments d'un tableau
 - `entries()` - parcourir les couples index/valeurs d'un tableau
 - `in` - parcourir les clés d'un tableau
 - `of` - parcourir les valeurs d'un tableau
 - `find()` - premier élément qui satisfait une condition
 - `findIndex()` - premier index qui satisfait une condition
 - `indexOf()` et `lastIndexOf()` - premier/dernier élément qui correspond
 - `push()`, `pop()`, `shift()` et `unshift()` - ajouter/supprime au début/fin dans un tableau
 - `slice()` - ne conserver que certaines lignes d'un tableau
 - `splice()` - supprimer/insérer/remplacer des valeurs dans un tableau
 - `concat()` - joindre deux tableaux
 - `join()` - joindre des chaînes de caractères
 - `keys()` et `values()` - les clés/valeurs d'un objet
 - `includes()` - vérifier si une valeur est présente dans un tableau
 - `every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau
 - `fill()` - remplir un tableau avec des valeurs
 - `flat()` - aplatir un tableau
 - `sort()` - pour trier un tableau
 - `map()` - tableau avec les résultats d'une fonction
 - `filter()` - tableau avec les éléments passant un test
 - `groupBy()` - regroupe les éléments d'un tableau selon un règle
 - `flatMap()` - chaînage de `map()` et `flat()`
 - `reduce()` et `reduceRight()` - réduire un tableau à une seule valeur
 - `reverse()` - inverser l'ordre du tableau
- Techniques
 - `` (backticks) - pour des expressions intelligentes
 - `new Set()` - pour supprimer les doublons
- Fonctions
 - Déclaration de fonction
 - Fonctions immédiatement invoquées (IIFE)
- Conclusion

Introduction

Votre introduction avec notamment les objectifs opérationnels du module.

Contenu du module

Introduction à la programmation fonctionnelle

- Paradigmes de programmation
- Fonctions fléchées
- Définition et utilité de la programmation fonctionnelle

Fonctions fondamentales

- `map()`
- `filter()`
- `reduce()`

Concepts de programmation fonctionnelle

- **First-class citizen**
- **Fonctions lambda**
- **Immuabilité**
- **Fonctions pures**
- **Composition de fonctions :**
 - Fonctions unaires
 - Currying
 - Closure
 - Fonction *pipe*
- **Récursion**
- **Builder pattern**
- **Refactorisation**

Opérateurs javascript super-coool 🕶️

opérateur `?:`

L'expression `question?valeur1:valeur2` retournera `valeur1` si `question` vaut `true` sinon elle retournera `valeur2`.

```
const age = 15;  
const resultat = age >= 18 ? 'majeur' : 'mineur'; // 'mineur'
```

opérateur `??`

Cet opérateur logique se nomme l'opérateur de "coalescence des nuls".

Renvoie son opérande de droite lorsque son opérande de gauche vaut `null` ou `undefined` et qui renvoie son opérande de gauche sinon.

```
const foo1 = null ?? 'default'; // "default"
const foo2 = 0 ?? 42; // 0
```

[!CAUTION] Contrairement à l'opérateur logique OU (`||`), l'opérande de gauche sera également renvoyé s'il s'agit d'une valeur équivalente à `false` et pas seulement `null` et `undefined`.

⚠ En d'autres termes **ATTENTION** !! lors de l'utilisation de `||` pour fournir une valeur par défaut à une variable, car on peut rencontrer des comportements inattendus lorsqu'on considère certaines valeurs comme correctes et utilisables (par exemple une chaîne vide `' '` ou `0`) !!

```
const foo3 = 0 || 42; // 42 => ATTENTION !
const foo4 = 1 || 42; // 1
const foo5 = null || 'salut !'; // 'salut !'
const foo6 = ' ' || 'salut !'; // 'salut !' => ATTENTION !
```

opérateur `??=`

Cet opérateur logique se nomme l'opérateur d'affectation de "coalescence des nuls", également connu sous le nom d'opérateur affectation logique nulle.

Évalue l'opérande de droite et l'attribue à gauche **UNIQUEMENT si l'opérande de gauche est nulle** (`null` ou `undefined`).

```
const a = { duration: 50 };
a.duration ??= 10; // pas fait
a.speed ??= 25; // fait => { duration: 50, speed: 25 }
```

opérateur de décomposition 'spread' `...`

L'opérateur de décomposition `spread` `...` permet de décomposer un itérable (comme un tableau) en ses éléments distincts. Cela permet de rapidement copier tout ou une partie d'un tableau existant dans un autre tableau ou d'en extraire facilement des parties.

```
// Combiner des valeurs existantes dans un nouveau tableau
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];

// Extraire uniquement ce qui est utile d'un tableau
const numbers = [1, 2, 3, 4, 5, 6];
const [one, two, ...rest] = numbers;

// Mariage d'objets avec mise à jour :- )
const myVehicle = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red',
};
const updateMyVehicle = {
  type: 'car',
  year: 2021,
  color: 'yellow',
};
const myUpdatedVehicle = { ...myVehicle, ...updateMyVehicle };
```

Déstructuration

L'opérateur de décomposition spread `...` sert aussi à isoler certains éléments afin de les utiliser ensuite, et de **mettre le reste** d'un coup ailleurs.

```
const valeurs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const [a, b, ...c] = valeurs;
console.log(a); // 1
console.log(b); // 2
console.log(c); // [3, 4, 5, 6, 7, 8, 9, 10]
```

Date et Heure

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Date

Obtenir la date et/ou heure actuelle

```
const maintenant = new Date(); // Obtenir l'un comme l'autre

console.log(maintenant.toLocaleDateString()); // ex: "06.06.2025"
console.log(maintenant.toLocaleTimeString()); // ex: "15:23:42"

const jour = maintenant.getDate();
const mois = maintenant.getMonth() + 1; // Attention : janvier = 0
const annee = maintenant.getFullYear();
const heure = maintenant.getHours();
const minute = maintenant.getMinutes();
const seconde = maintenant.getSeconds();
console.log(`${jour}/${mois}/${annee} - ${heure}h${minute}`);

// Au format ISO (standard international)
console.log(maintenant.toISOString()); // ex: "2025-06-06T13:23:42.123Z"
```

Math

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math

Math.PI - la constante π

Description

Math.PI fournit la valeur numérique de π (approximativement 3.14159). Utile pour tout calcul géométrique impliquant des cercles (périmètre, surface, volumes de révolution).

Exemple : calculer la circonférence et l'aire d'un cercle.

```
const r = 3; // rayon
const circonference = 2 * Math.PI * r; // P = 2πr
const aire = Math.PI * Math.pow(r, 2); // A = πr²
console.log({ circonference, aire });
```

Math.abs() - la |valeur absolue| d'un nombre

Description

Renvoie la valeur absolue (distance à zéro) d'un nombre. Pratique pour normaliser des différences, calculer des écarts ou éviter des signes indésirables.

```
Math.abs(-5); // 5  
Math.abs(3 - 10); // 7
```

Math.pow() - élever à une puissance

Description

Élève un nombre à la puissance donnée. Depuis ES2016 on peut aussi utiliser l'opérateur ** (ex: x ** y).

```
Math.pow(2, 3); // 8  
2 ** 3; // 8
```

Math.min() - plus petite valeur

Description

Retourne la plus petite valeur parmi les arguments fournis. Utile pour bornes, validations et comparaisons rapides.

```
Math.min(3, 7, -1, 0); // -1  
Math.min(...[10, 2, 8]); // 2
```

Math.max() - plus grande valeur

Description

Retourne la plus grande valeur parmi les arguments fournis.

```
Math.max(3, 7, -1, 0); // 7  
Math.max(...[10, 2, 8]); // 10
```

Math.ceil() - arrondir à la prochaine valeur entière la plus proche

Description

Renvoie le plus petit entier supérieur ou égal au nombre — arrondi « vers le haut ».

```
Math.ceil(3.14); // 4
Math.ceil(-1.2); // -1
```

Math.floor() - arrondir à la précédente valeur entière la plus proche

Description

Renvoie le plus grand entier inférieur ou égal au nombre — arrondi « vers le bas ».

```
Math.floor(3.9); // 3
Math.floor(-1.2); // -2
```

Math.round() - arrondir à la valeur entière la plus proche

Description

Arrondit au nombre entier le plus proche. Les valeurs .5 sont arrondies vers l'entier pair suivant l'implémentation JS courante (arrondit vers $+\infty$ pour .5 dans la plupart des moteurs).

```
Math.round(3.2); // 3
Math.round(3.5); // 4
```

Math.trunc() - supprime la virgule et retourne la partie entière d'un nombre

Description

Renvoie la partie entière en supprimant la fraction (troncature). Contrairement à floor, trunc ne dépend pas du signe.

```
Math.trunc(3.9); // 3
Math.trunc(-1.9); // -1
```

Math.sqrt() - la racine carrée d'un nombre

Description

Renvoie la racine carrée d'un nombre. Utile pour calculs géométriques, distances, normalisations.

```
Math.sqrt(9); // 3
Math.sqrt(2); // ~1.4142
```

Math.random() - générer un nombre aléatoire entre 0.0 (compris) et 1.0 (non compris)

Description

Retourne un flottant pseudo-aléatoire dans [0, 1]. Pour obtenir un entier dans un intervalle, combiner avec Math.floor/ceil.

```
// entier aléatoire entre min (inclus) et max (inclus)
function randint(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}

randint(1, 6); // 1..6
```

JSON

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/JSON

JSON.stringify() - transformer un objet Javascript en JSON

Description

Transforme un objet JS en une chaîne JSON. Pratique pour stocker ou envoyer des données sur le réseau (API, stockage local).

```
const obj = { nom: 'Cyril', age: 20, lang: ['JS', 'Python'] };
const json = JSON.stringify(obj);
console.log(json); // '{"nom":"Cyril","age":21,"lang":["JS","Python"]}'
```

JSON.parse() - transformer du JSON en objet Javascript

Description

Transforme une chaîne JSON en objet Javascript. Attention aux données non fiables : valider ou utiliser un parseur sécurisé si nécessaire.

```
const json = '{"nom":"Cyril","age":21}';  
const obj = JSON.parse(json);  
console.log(obj.nom); // 'Cyril'
```

Chaînes de caractères

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String

split() - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau

Description

Découpe une chaîne en un tableau selon un séparateur (caractère, expression régulière). Très utile pour parser des CSV simples ou des entrées utilisateurs.

```
'a,b,c'.split(','); // ['a','b','c']  
'2025-10-28'.split('-'); // ['2025','10','28']
```

trim(), trimStart() et trimEnd() - épuration des espaces en trop dans une chaîne (trimming)

Description

Supprime les espaces superflus en début/fin de chaîne. Indispensable pour normaliser des saisies utilisateur.

```
' hello '.trim(); // 'hello'  
' hello '.trimStart(); // 'hello '  
' hello '.trimEnd(); // ' hello'
```

`padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères

Description

Ajoutent des caractères au début ou à la fin pour atteindre une longueur donnée (utile pour formatage, tableaux, affichage en console).

```
'5'.padStart(3, '0'); // '005'  
'42'.padEnd(4, ' ').replace(/ /g, '\u00B7'); // '42..'
```

Console

Lien vers la documentation officielle : <https://developer.mozilla.org/fr/docs/Web/API/console>

`console.log()` - Afficher un message sur la console

```
console.log('Coucou !'); // Coucou !
```

`console.info()`, `warn()` et `error()` - Afficher un message sur la console (filtrables)

Description

Ces variantes de console permettent de catégoriser les messages (info, warning, error). En dev on s'en sert pour signaler états, avertissements et erreurs sans polluer la sortie principale.

```
console.info('Chargement OK');  
console.warn('Temps de réponse élevé');  
console.error("Impossible de se connecter à l'API");
```

`console.table()` - Afficher tout un tableau ou un objet sur la console

Description

Affiche un tableau structuré en colonnes — très pratique pour inspecter des tableaux d'objets en dev.

```
console.table([
  { nom: 'Alice', age: 23 },
  { nom: 'Bob', age: 25 },
]);
```

`console.time()`, `timeLog()` et `timeEnd()` - Chronométrer une durée d'exécution

Description

Permettent de mesurer des durées côté client/serveur pour repérer des goulets d'étranglement.

```
console.time('boucle');
for (let i = 0; i < 1e6; i++); // travail
console.timeEnd('boucle'); // 'boucle: XXms'
```

Tableaux

Lien vers la documentation officielle :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array

`forEach` - parcourir les éléments d'un tableau

Description

Parcourt chaque élément du tableau et exécute une fonction. Ne renvoie rien (utiliser `map`/`filter`/`reduce` si on veut transformer/reduire).

```
[1, 2, 3].forEach((x) => console.log(x));
```

`entries()` - parcourir les couples index/valeurs d'un tableau

Description

Renvoie un itérateur de paires [index, valeur], utile avec for..of pour parcourir les indices et valeurs.

```
for (const [i, v] of ['a', 'b', 'c'].entries()) {  
  console.log(i, v);  
}
```

in - parcourir les clés d'un tableau

Description

L'opérateur **in** teste si une clé existe dans un objet/array. Pour parcourir les indices d'un tableau on peut l'utiliser dans une boucle for.

```
console.log(2 in ['a', 'b', 'c']); // true  
for (const i in ['a', 'b', 'c']) console.log(i); // '0','1','2'
```

of - parcourir les valeurs d'un tableau

Description

for..of itère sur les valeurs d'un itérable (tableaux, strings, maps...). Pratique et lisible.

```
for (const v of ['a', 'b', 'c']) console.log(v);
```

find() - premier élément qui satisfait une condition

Description

Retourne le premier élément qui satisfait la fonction de test ou undefined.

```
[{ id: 1 }, { id: 2 }].find((x) => x.id === 2); // {id:2}
```

findIndex() - premier index qui satisfait une condition

Description

Retourne l'index du premier élément qui passe la condition, ou -1 si aucun.

```
[10, 20, 30].findIndex((x) => x > 15); // 1
```

indexOf() et lastIndexOf() - premier/dernier élément qui correspond

Description

indexOf renvoie le premier index correspondant; lastIndexOf le dernier. Pour arrays d'objets, utiliser findIndex.

```
[1, 2, 3, 2].indexOf(2); // 1
[1, 2, 3, 2].lastIndexOf(2); // 3
```

push(), pop(), shift() et unshift() - ajouter/supprime au début/fin dans un tableau

Description

push ajoute à la fin, pop retire à la fin; unshift ajoute au début, shift retire au début. Mutent le tableau.

```
const t = [1, 2];
t.push(3); // [1,2,3]
t.pop(); // 3, t => [1,2]
t.unshift(0); // [0,1,2]
t.shift(); // 0, t => [1,2]
```

slice() - ne conserver que certaines lignes d'un tableau

Description

slice renvoie une copie superficielle d'une portion du tableau — non destructive.

```
[1, 2, 3, 4].slice(1, 3); // [2,3]
```

splice() - supprimer/insérer/remplacer des valeurs dans un tableau

Description

splice modifie le tableau en place: suppression, insertion ou remplacement selon les arguments.

```
const a = [1, 2, 3, 4];  
a.splice(1, 2, 9, 9); // supprime 2 éléments à l'index 1, insère 9,9 => a =  
[1,9,9,4]
```

concat() - joindre deux tableaux

Description

Retourne un nouveau tableau résultat de la concaténation (non destructif).

```
[1, 2].concat([3, 4]); // [1,2,3,4]  
[...a, ...b]; // équivalent moderne
```

join() - joindre des chaînes de caractères

Description

Rejoint les éléments d'un tableau en une seule chaîne séparée par le séparateur fourni.

```
['a', 'b', 'c'].join('-'); // 'a-b-c'
```

keys() et values() - les clés/valeurs d'un objet

Description

Sur un tableau, keys() renvoie un itérateur d'indices, values() d'éléments. Sur Map, ces méthodes sont aussi utiles pour itérer.

```
for (const k of ['a', 'b'].keys()) console.log(k); // 0,1  
for (const v of ['a', 'b'].values()) console.log(v); // 'a','b'
```

includes() - vérifier si une valeur est présente dans un tableau

Description

Retourne true si la valeur est présente, false sinon. Pour objets, comparer par référence.

```
[1, 2, 3].includes(2); // true
```

`every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau

Description

`every` renvoie `true` si tous les éléments satisfont le test; `some` renvoie `true` si au moins un satisfait.

```
[2, 4, 6].every((x) => x % 2 === 0); // true
[1, 2, 3].some((x) => x > 2); // true
```

`fill()` - remplir un tableau avec des valeurs

Description

Remplit un tableau existant avec la valeur fournie (mutation). Utile pour initialiser des buffers ou tests.

```
new Array(3).fill(0); // [0,0,0]
```

`flat()` - aplatir un tableau

Description

Applatit les tableaux imbriqués jusqu'à une profondeur donnée (par défaut 1).

```
[1, [2, 3], [4, [5]]].flat(); // [1,2,3,4,[5]]
[1, [2, [3]]].flat(2); // [1,2,3]
```

`sort()` - pour trier un tableau

Description

Trie un tableau en place. Par défaut trie les éléments comme des strings; pour nombres fournir une fonction de comparaison.

```
[3, 1, 2].sort((a, b) => a - b); // [1,2,3]
['b', 'a'].sort(); // ['a','b']
```


map() - tableau avec les résultats d'une fonction

Description

Crée un nouveau tableau contenant le résultat de l'appel d'une fonction sur chaque élément. Pur et non mutatif.

```
[1, 2, 3].map((x) => x * 2); // [2,4,6]
```

filter() - tableau avec les éléments passant un test

Description

Renvoie un nouveau tableau avec les éléments qui satisfont la condition donnée.

```
[1, 2, 3, 4].filter((x) => x % 2 === 0); // [2,4]
```

groupBy() - regroupe les éléments d'un tableau selon un règle

Description

Cette méthode (selon l'environnement) regroupe les éléments selon une clé. En pratique on utilise reduce pour compatibilité.

```
const animaux = [
  { type: "mammifère", nom: "chien" },
  { type: "oiseau", nom: "moineau" },
  { type: "mammifère", nom: "chat" },
  { type: "poisson", nom: "saumon" },
];

// On regroupe les animaux par type
const groupes = Object.groupBy(animaux, (a) => a.type);

console.log(groupes);
//Résultat
{
  mammifère: [
    { type: "mammifère", nom: "chien" },
    { type: "mammifère", nom: "chat" }
  ],
  oiseau: [
```

```

    { type: "oiseau", nom: "moineau" }
  ],
  poisson: [
    { type: "poisson", nom: "saumon" }
  ]
}

```

flatMap() - chaînage de map() et flat()

Description

Applique une fonction à chaque élément et aplatit le résultat d'un niveau. Pratique pour transformer et aplatir en un seul passage.

```
[1, 2].flatMap((x) => [x, x * 2]); // [1,2,2,4]
```

reduce() et reduceRight() - réduire un tableau à une seule valeur

Description

Réduit un tableau à une seule valeur en appliquant une fonction accumulateur. Très puissant : sommes, groupements, transformations complexes. Exemple 1 :

```
[1, 2, 3].reduce((acc, x) => acc + x, 0); // 6
```

Exemple 2 :

```

const produits = [
  { categorie: 'fruit', nom: 'pomme' },
  { categorie: 'légume', nom: 'carotte' },
  { categorie: 'fruit', nom: 'banane' },
];

const groupés = produits.reduce((acc, p) => {
  // Si la catégorie n'existe pas encore, on la crée
  if (!acc[p.categorie]) {
    acc[p.categorie] = [];
  }
  // On ajoute le produit dans la bonne catégorie
  acc[p.categorie].push(p.nom);

```

```
    return acc;
  }, {}));

console.log(groupés);
// → { fruit: ["pomme", "banane"], légume: ["carotte"] }
```

reverse() - inverser l'ordre du tableau

Description

Inverse le tableau en place. Attention : mutation.

```
const a = [1, 2, 3];
a.reverse(); // a => [3,2,1]
```

Techniques

`(backticks) - pour des expressions intelligentes

Description

Les backticks (template literals) permettent d'insérer des expressions et d'écrire des strings multi-lignes facilement. Très pratiques pour construire des messages ou des templates.

```
const nom = 'Cyril';
const msg = `Bonjour ${nom}, aujourd'hui nous sommes ${new
Date().toLocaleDateString()}`;
```

new Set() - pour supprimer les doublons

Description

Set est une structure qui stocke des valeurs uniques. Utile pour dédupliquer rapidement un tableau.

```
const arr = [1, 2, 2, 3];
const dedup = [...new Set(arr)]; // [1,2,3]
```

Fonctions

Déclaration de fonction

Standard

```
function doStuff(a, b, c) {  
  return a + b + c;  
}
```

Sous forme d'expression de fonction

```
const doStuff = function (a, b, c) {  
  return a + b + c;  
};
```

Sous forme d'expression de fonction anonyme

```
const doStuff = (a, b, c) => {  
  return a + b + c;  
};
```

Sous forme raccourcie

S'il n'y a qu'un seul argument et que son corps n'a qu'une seule expression, on peut omettre le return et le corps de la fonction :

```
const doStuff = (a) => `Salut ${a} !`;
```

Fonctions immédiatement invoquées (IIFE)

IIFE = Immediately Invoked Function Expressions.

Ces fonctions sont définies et **exécutées immédiatement**. Elles sont souvent utilisées pour créer un **contexte isolé** ou encapsuler du code sans polluer l'espace global.

```
(function(){ ... })()
```

ou

```
(( ) => { ... })()
```

Conclusion

Ce module 323 m'a vraiment appris une nouvelle façon de penser le code : la programmation fonctionnelle. Au début, ce n'était pas facile de changer mes habitudes, car j'avais tendance à coder de manière plus « classique », avec des boucles et des conditions. Mais petit à petit, en pratiquant, j'ai commencé à comprendre l'intérêt d'utiliser des fonctions comme `map()`, `filter()` ou `reduce()`.

Ces outils m'ont permis d'écrire du code plus simple, plus propre et souvent plus efficace. J'ai aussi compris que la programmation fonctionnelle, ce n'est pas juste une autre manière de faire la même chose, mais une façon différente de structurer sa logique pour que le code soit plus clair.

Certaines notions, comme `reduce()`, m'ont pris un peu plus de temps à comprendre, mais avec les exercices et les exemples du cours, ça a fini par faire sens. Maintenant, j'arrive mieux à transformer ou regrouper des données sans forcément passer par des boucles compliquées.

En résumé, ce module m'a vraiment fait progresser. Il m'a appris à réfléchir autrement, à mieux utiliser les outils de JavaScript et à écrire du code plus réfléchi. Je sais que j'ai encore des choses à améliorer, mais je sens déjà une vraie évolution dans ma façon de coder.

Votre conclusion avec les éléments usuels